Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Comprehensive, point-wise checklist for a **PostgreSQL Health Check** — designed to help you ensure optimal performance, availability, and reliability.

🧩 PostgreSQL Health Check — Key Steps for Optimal Performance

When performing a **PostgreSQL database health check**, I follow these steps to ensure the system is healthy, performant, and secure:

1 Verify PostgreSQL Services

- Confirm that the PostgreSQL service is up and running (systemctl status postgresql or pg_ctl status).
- Check the uptime and ensure there are no recent restarts or crashes.

2 Review PostgreSQL Logs

- Inspect the PostgreSQL log files (usually under /var/lib/pgsql/data/pg_log/ or /var/log/postgresql/) for:
  - Connection errors
  - Autovacuum or checkpoint warnings
  - Deadlocks and failed transactions
  - Slow query logs

3 Check Server Resource Usage

- Review system-level metrics:
  - **CPU usage** (top, htop, or vmstat)
  - **Memory usage** and swap activity
  - **Disk I/O performance** (iostat, sar, iotop)
  - **Disk space** available for data, WAL, and temp directories

4 Validate Database and Cluster Status

- Check all databases in the cluster with \l (psql).
- Ensure no databases are in recovery or read-only unexpectedly.
- Confirm that all schemas and tablespaces are accessible.

5 Backup & Restore Validation

- Verify recent backup completion using tools like **pg_dump**, **pg_basebackup**, or **pgBackRest**.
- Perform a **test restore** to confirm data integrity and recoverability.

6 Check WAL and Replication

- Monitor **WAL (Write-Ahead Log)** directory growth and archiving status.
- If replication is configured:
  - Run SELECT * FROM pg_stat_replication; to check lag and status.
  - Verify archive_command and restore_command success.

7 Inspect Autovacuum & Table Bloat

- Check autovacuum activity and recent runs:
  - Query pg_stat_all_tables for last_vacuum, last_autovacuum, n_dead_tup.
- Identify bloated tables and indexes using pgstattuple or pg_bloat_check.sql.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

8 Analyze Query Performance

- Review slow queries from pg_stat_statements.
- Identify top resource-consuming SQL statements.
- Check indexes usage (idx_scan, seq_scan ratios).
- Look for missing indexes and unused ones.

9 Check Connection Health

- Validate current connections with SELECT * FROM pg_stat_activity;.
- Identify idle or long-running connections.
- Review max_connections and connection pooling configuration (PgBouncer, Pgpool-II).

10 Review Configuration Parameters

- Validate key performance-related parameters:
  - shared_buffers, work_mem, maintenance_work_mem
  - effective_cache_size, checkpoint_timeout, wal_buffers
  - autovacuum_max_workers, max_wal_size
- Ensure tuning matches system hardware and workload.

11 Validate Security & Access Controls

- Check pg_hba.conf and postgresql.conf for proper access rules.
- Review user roles, privileges, and password policies.
- Monitor failed login attempts and orphaned roles.

12 Monitor Replication & High Availability

- If using **Streaming Replication**, ensure replicas are synchronized.
- For **Patroni, Pgpool-II, or EDB Failover Manager** setups — verify leader election, failover logs, and node status.

13 Review Alerts & Monitoring Dashboards

- Validate integration with tools like:
  - **pgAdmin**, **Zabbix**, **Prometheus + Grafana**, **pgwatch2**
- Ensure alerting thresholds (CPU, replication lag, failed backups) are active.

✅ Final Validation

- Run a **comprehensive health report** summarizing:
  - Resource usage trends
  - Performance statistics
  - Backup and replication status
  - Security findings
  - Configuration deviations